

INDEX

SR. NO.	TITLE	PAGE NO.	PERFORMANCE DATE	ASSESSMENT DATE	MARKS	SIGN.
1	Define and objective of the specific module and perform Required Engineering Process.					
2	Identify suitable Design and Implementation Model from the different Software Engineering Models. Study Software Requirement Engineering.					
3	Student should include SRS document for current semester project.					
4	Develop Software Project Management Planning (SPMP) for specified module.					
5	Cost and Effort Estimation using Software Cost Estimation Models.					
6	To prepare a system analysis and design for the Peer-to-Peer Lodging Exchange platform using structured design techniques such as Data Flow Diagrams(DFD) and Data Dictionary.					
7	To design a Peer-to-Peer Lodging Exchange module using an Objective-Oriented Approach(OOA) by creating Use Case Diagrams with scenarios, Class Diagrams, State Diagram, Collaboration Diagrams, Sequence Diagrams, and Activity Diagrams.					

8	Defining Coding Standards and walk through.					
9	Write the test cases for the identified module.					
10	Demonstrate the use of different Testing Tools with comparison.					
11	Define security and quality aspects of the identified module					

Practical No. 1

Aim:-

Define and objective of the specific module and perform Required Engineering Process.

Project Title:- AI Mock Test Generator — CBT Exam Simulator

Parameters:-

1. Context

The AI Mock Test Generator — CBT Exam Simulator is a web-based application aimed at solving the real-world problem faced by aspirants preparing for Indian competitive examinations such as SBI PO, IBPS PO, Railway RRB NTPC, SSC CGL and SSC CHSL. Conventional mock-test platforms depend on manually prepared, static question banks that rarely adapt to a student's specific subject, chapter, difficulty level or personal study material. This project addresses that gap by using the Google Gemini Generative AI API to dynamically generate multiple-choice questions on demand and by reproducing a realistic Computer-Based Test (CBT) environment — complete with a countdown timer, question palette, mark-for-review, negative marking and detailed analytics — so that aspirants can practise repeatedly under exam-like conditions without depending on a fixed, pre-written question bank.

2. Information

AI Mock Test Generator is a Python and Streamlit web application. The user first configures a test by choosing a target examination (or a custom exam name), a mock-test format (Full Mock, Sectional or Chapter-wise), one or more subjects, a chapter/topic where applicable, a difficulty level, the number of questions, the exam duration and whether negative marking should be applied. The user may optionally upload reference material in PDF, DOCX or TXT format — which the system extracts using pdfplumber and python-docx and truncates to approximately 15,000 characters to manage Gemini API token usage — or request an AI-generated syllabus and question-pattern research summary instead. This configuration, together with any reference material, is sent to the Gemini Generative AI API, which returns a structured JSON set of MCQs, each carrying an identifier, section, question text, four options, the correct answer, an explanation and a

difficulty tag. The application then loads these questions into a full CBT examination interface built with Streamlit session state, runs a live countdown timer using streamlit-autorefresh, records every answer along with its visited / marked-for-review state, evaluates the attempt on submission (manual or automatic on time-out), computes the score, accuracy and a section-wise breakdown, displays detailed explanations for every question, and finally stores a summary of the attempt in a local SQLite database (mocktest_history.db) so that the aspirant can review historical performance and a score-progression chart.

3. Objectives

The Software Engineering process for this module is guided by the following objectives:

1. To develop an AI-powered mock-test generation system that removes the dependency on a static, manually maintained question bank.
2. To generate multiple-choice questions dynamically using the Google Gemini Generative AI API based on exam, subject, chapter, difficulty and reference material.
3. To support multiple predefined Indian competitive examinations (SBI PO, IBPS PO, Railway RRB NTPC, SSC CGL, SSC CHSL) as well as a user-defined custom examination.
4. To allow the aspirant to choose between Full Mock, Sectional and Chapter-wise test formats.
5. To allow study material to be supplied as PDF, DOCX or TXT, or to request an AI-generated syllabus and pattern research summary instead.
6. To provide a realistic, timed CBT examination interface with a question palette, answer marking, clear-response and mark-for-review functionality.
7. To implement optional, configurable negative marking and compute the score, accuracy and section-wise performance automatically.
8. To provide AI-generated explanations for every question and maintain historical results locally using SQLite.

4. Functions

The configuration function collects the examination, mock-test type, subject(s), chapter/topic, difficulty, question count, duration and negative-marking preference from the aspirant and validates them before generation begins. The material-processing function extracts text from uploaded PDF, DOCX or TXT files (or triggers a Gemini-based syllabus research call when no material is supplied) and prepares a bounded reference string. The question-generation function builds a structured prompt from the configuration and reference text, calls the Gemini Generative AI API, and parses the returned JSON into individual Question objects, retrying or reporting an error if the response is malformed. The examination function renders the CBT interface — timer, question palette, options and navigation controls — and maintains the aspirant's answers, visited state and mark-for-review flags in Streamlit session state until the test is submitted or the timer expires. The scoring function evaluates every answer against the correct option, applies the configured negative-marking penalty, and computes the final score, accuracy and a section-wise breakdown. The history function writes a summary record of the attempt to the local SQLite database and later retrieves it to display previous attempts and a score-progression chart.

5. Performance

The exam-configuration screen must respond immediately to user input, since no server round-trip is required at this stage. Question generation is bounded chiefly by the response time of the Gemini Generative AI API and by network conditions; the interface displays a clear loading state while this call is in progress and offers a Try Again option if generation fails. Once questions are loaded, navigation between questions, answer selection, marking for review and palette updates must feel instantaneous, since these operations only update local Streamlit session state. The live countdown timer must refresh every second using streamlit-autorefresh without perceptibly slowing down the rest of the interface. Score calculation, section-wise analysis and the writing of a result record to SQLite must complete within a short, unnoticeable interval immediately after submission.

6. Problem Decomposition

The overall problem is divided into five sub-problems that map directly onto the modules of the system:

- Test configuration — collecting and validating exam, mock-type, subject, chapter, difficulty, question-count, duration and negative-marking parameters.
- Reference-material acquisition — extracting text from PDF / DOCX / TXT uploads or generating an AI research summary of syllabus and question pattern.
- AI question generation — designing an effective Gemini prompt, requesting a structured JSON response, and parsing / validating that response into Question objects, with retry on failure.
- CBT examination engine — building the timed, session-state-driven exam interface with question palette, answer management (Save & Next, Mark for Review & Next, Clear Response, Save & Mark for Review) and automatic submission on time-out.

Scoring, analytics and history — computing score and accuracy, generating section-wise and detailed-solution views, and persisting a summary of every attempt to the local SQLite database for later review.

Requirement Tasks:-

1. Inception

The project began with the observation that competitive-exam aspirants repeatedly practise using the same limited set of static question banks, and that a generative AI model could remove this constraint by producing fresh, syllabus-aligned questions on demand. The developer evaluated technical feasibility by reviewing the Google Gemini Generative AI API documentation, the constraints of running a live countdown timer inside a Streamlit app via session state and streamlit-autorefresh, and the suitability of pdfplumber / python-docx for extracting study material. The inception phase concluded with a defined project idea, a chosen technology stack (Python, Streamlit, Gemini API, SQLite) and a preliminary feature list.

2. Elicitation

Requirements were gathered through self-study of real CBT interfaces used by banking, railway and SSC examinations, and by reviewing the expectations aspirants typically have of a mock-test platform: realistic timing, a question palette, mark-for-review, negative

marking and post-exam analytics. Functional requirements identified include exam and mock-type selection, subject/chapter configuration, study-material upload or AI research, AI-based question generation, a full CBT interface, scoring with optional negative marking, detailed explanations and test history. Non-functional requirements include responsiveness of the configuration screen, reliability of score calculation, portability across operating systems that support Python and Streamlit, and secure handling of the Gemini API key through environment variables. Constraints noted include the Gemini API's rate limits and token costs, and the roughly 15,000-character cap placed on extracted reference material. 3.

Elaboration

Each requirement was expanded into a detailed specification. The question-generation specification defines the exact Gemini prompt template, the required JSON schema (id, section, question, four options, correct answer, explanation, difficulty) and the parsing logic used to convert that JSON into Question objects. The CBT-interface specification defines the session-state variables (screen, view_mode, config, source_text, questions, answers, current_q_idx, start_time, confirm_submit, test_saved) and the palette states (not visited, visited but unanswered, answered, marked for review, answered and marked). The scoring specification defines the exact formulas: $\text{Final Score} = \text{Correct Answers} - (\text{Incorrect Answers} \times \text{Negative Mark Value})$, and $\text{Accuracy} = \text{Correct Answers} / \text{Attempted Questions} \times 100$. The database specification defines the results table used for test history, with columns for id, exam, subject, mock_type, score, total_questions, accuracy and date_taken.

4. Negotiation

After reviewing the full feature list against the available development time and the limitations of a single-user, local Streamlit deployment, several features were deliberately excluded from the current version. These include user registration and login, admin accounts, a cloud-hosted multi-user database, online payment, social login, live multiplayer examinations, webcam-based proctoring and advanced anti-cheating mechanisms. The features retained for this version are exam configuration, study-material processing, AI question generation, the CBT examination engine, scoring and analytics, and local SQLite- based test history. This scope reduction keeps the project deliverable within the academic timeline while still demonstrating the complete, realistic mock-test experience end to end.

5. Specification

The formal specification covers four areas. The configuration specification defines every parameter the aspirant can set before generation (examination, mock type, subjects, chapter/topic, difficulty, question count 5–100 in steps of 5, duration 5–180 minutes in steps of 5, and a negative-marking value from 0 to 2.0, defaulting to 0.25). The AI specification defines the Gemini prompt format and the expected JSON response schema. The CBT-interface specification includes the layout of the question screen (current question number, total questions, difficulty tag, question statement, four options, timer and navigation palette) and the four answer-management actions. The database specification defines the results table schema described above, stored in `mocktest_history.db`.

6. Validation

Validation is carried out through functional testing of each module individually (configuration validation, material extraction, Gemini question generation and JSON parsing, timer behaviour, scoring formulas), integration testing of the complete flow from configuration through to result and history, and informal user-acceptance testing by attempting full mock tests across more than one examination and difficulty combination. Each test verifies that the system behaves according to its specification, including correct negative-marking deductions, accurate accuracy calculation, proper palette-state transitions and a correctly saved history record. Defects such as malformed Gemini JSON responses are handled by surfacing a Try Again option rather than crashing the session.

7. Requirements Management

All requirements are tracked against the functional requirement list (FR-01 to FR-24, detailed in Practical 3) with a status and any noted change. Changes discovered during implementation — for example, the decision to cap extracted study material at approximately 15,000 characters to control Gemini API token usage — are recorded as amendments with a reason. Each implemented feature is linked back to its originating requirement to maintain traceability, and this requirement list also serves as the checklist used during the validation phase described above.

Conclusion:

AI Mock Test Generator — CBT Exam Simulator is a web-based application that combines the Google Gemini Generative AI API with a realistic Computer-Based Test interface to give competitive-exam aspirants a flexible, repeatable practice environment. The project demonstrates the practical application of software engineering principles across the complete lifecycle, from inception and requirement elicitation through specification, negotiation and validation. It integrates AI-driven content generation, timed examination logic, scoring analytics and local persistence into a single functional application that solves a genuine, everyday problem faced by exam aspirants, while deliberately scoping out multi- user and account-based features to remain deliverable within the current academic timeline.

Practical No.2

Aim:

Identify suitable Design and Implementation Model from the different Software Engineering Models.

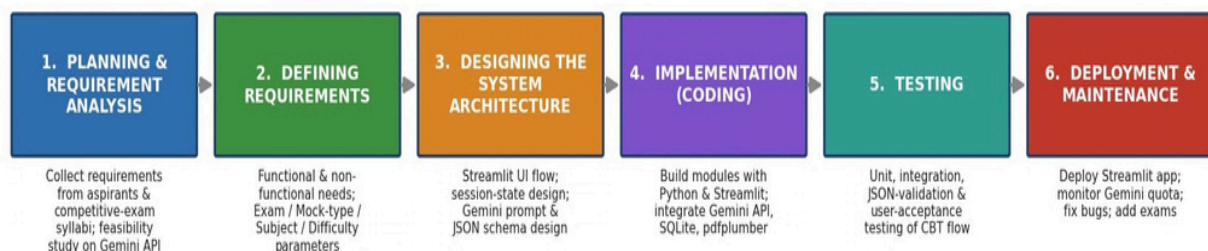
Software Process Models:

A Software Process Model is an abstract representation of the software development process, also known as the Software Development Life Cycle (SDLC). It defines a sequence of activities required to develop high-quality software in a systematic manner. A software process model helps developers to understand project requirements, plan development activities, reduce development risks, improve software quality and deliver the project on time. For the AI Mock Test Generator, different SDLC models can be evaluated depending on the project's characteristics—in particular, its dependence on an external generative AI service whose output quality can only be judged once real questions are generated and attempted.

A software process model helps developers to:

- Understand project requirements.
- Plan development activities.
- Reduce development risks.
- Improve software quality.
- Deliver the project on time.

SDLC PHASES – AI MOCK TEST GENERATOR (CBT EXAM SIMULATOR)



SDLC Phases for AI Mock Test Generator

Stage 1: Planning and Requirement Analysis

In this phase, requirements are collected by studying real CBT interfaces used by SBI PO, IBPS PO, Railway RRB NTPC, SSC CGL and SSC CHSL examinations, and by identifying the parameters an aspirant needs to configure a meaningful mock test.

Requirements Collected:

- Exam and custom-exam selection
- Mock-test type (Full Mock / Sectional / Chapter-wise)
- Subject and chapter/topic selection
- Study-material upload or AI syllabus research
- Difficulty, question-count and duration configuration
- Negative marking, CBT interface, timer and question palette
- Scoring, analytics, explanations and test history

A feasibility study is also performed to confirm that the project is technically feasible (Gemini API availability and quota), economically feasible (no licensing cost beyond API usage) and operationally feasible (a single-user Streamlit deployment is sufficient for the intended use case).

Stage 2: Defining Requirements

All requirements are documented in the Software Requirement Specification (SRS), detailed fully in Practical 3. The SRS includes:

Functional Requirements:

- Exam & configuration management
- Study material upload / AI research
- AI-based question generation
- CBT examination interface
- Answer management & navigation
- Scoring, accuracy & section-wise analytics
- Test history

Non-functional Requirements:

- Performance

- Usability
- Reliability
- Maintainability
- Portability
- Security

Stage 3: Designing the Product Architecture

The architecture of the AI Mock Test Generator is designed around independent, loosely-coupled modules so that features can be built and released incrementally.

Main Modules:

- Exam & Configuration Module
- Study Material / AI Research Module
- AI Question Generation Module
- CBT Examination Module
- Answer Management Module
- Timer Module
- Scoring & Analytics Module
- Test History Module (SQLite)

Persisted Data:

- results (id, exam, subject, mock_type, score, total_questions, accuracy, date_taken)

Stage 4: Implementation (Coding)

The AI Mock Test Generator is developed using:

<i>Component</i>	<i>Technology</i>
Frontend / UI	Streamlit (Python)
Backend Logic	Python
Generative AI	Google Gemini Generative AI API
Database	SQLite
PDF / DOCX Extraction	pdfplumber / python-docx
Configuration	python-dotenv (.env)
Timer Refresh	streamlit-autorefresh

<i>Component</i>	<i>Technology</i>
IDE	VS Code

Coding is done module-wise, for example: Exam Configuration Module, AI Question Generation Module, CBT Examination Module, Scoring Module, Test History Module.

Stage 5: Testing

Each module is tested individually.

Testing includes:

- Unit Testing
- Integration Testing
- System Testing
- User Acceptance Testing

Modules Tested:

- Exam & configuration validation
- Study-material extraction (PDF / DOCX / TXT)
- AI question generation & JSON parsing
- CBT navigation, timer & auto-submit
- Scoring, negative marking & accuracy
- Test-history persistence & retrieval

Stage 6: Deployment and Maintenance

After successful testing, the AI Mock Test Generator is deployed as a Streamlit application.

Maintenance activities include:

- Fixing bugs found during use
- Monitoring Gemini API quota and handling API errors gracefully
- Adding new predefined examinations
- Database backup of mocktest_history.db
- Improving prompt quality for better generated questions

List of Software Process Models

- Waterfall Model
- Prototype Model
- RAD Model
- Incremental Model

- Spiral Model
- Agile Model
- V-Model
- Component-Based Model

Difference Between Software Models

<i>Model</i>	<i>Main Idea</i>	<i>Advantages</i>	<i>Disadvantages</i>	<i>Suitable for Mock Test Generator</i>
Waterfall	Linear development	Easy to understand	Difficult to change requirements	✓✓
Prototype	Prototype first	User feedback	More development time	✓✓✓
Incremental	Module-wise development	Early releasable versions; easy to isolate AI-related risk	Needs careful interface planning between increments	✓✓✓✓✓
Spiral	Risk analysis	High quality	Expensive, needs risk-analysis expertise	✓✓
Agile	Sprint-based development	Flexible	Requires experienced team	✓✓✓✓
V-Model	Development with parallel testing	Better quality	Less flexible	✓✓

Diagram for Selected Model and its Description:

❖ *Incremental Model*

The Incremental Model divides the required functionality into a series of increments, each of which is analysed, designed, coded and tested like a small waterfall, and each of which

delivers a working, releasable version of the system. This model was selected over a single-pass Waterfall model chiefly because the quality of AI-generated questions cannot be fully judged until real questions are generated, attempted and reviewed — an early, working increment lets this be evaluated and refined before the more exam-realistic and analytical features are layered on top.

When to use?

- The system can be naturally divided into independent functional increments
- Core functionality needs to be delivered and demonstrated early
- Some requirements (for example, AI prompt design) are expected to be refined after initial feedback
- The team wants to reduce risk by validating the riskiest part of the system (Gemini integration) in an early increment
- The project has a fixed academic timeline but benefits from iterative refinement

INCREMENTAL MODEL — AI MOCK TEST GENERATOR

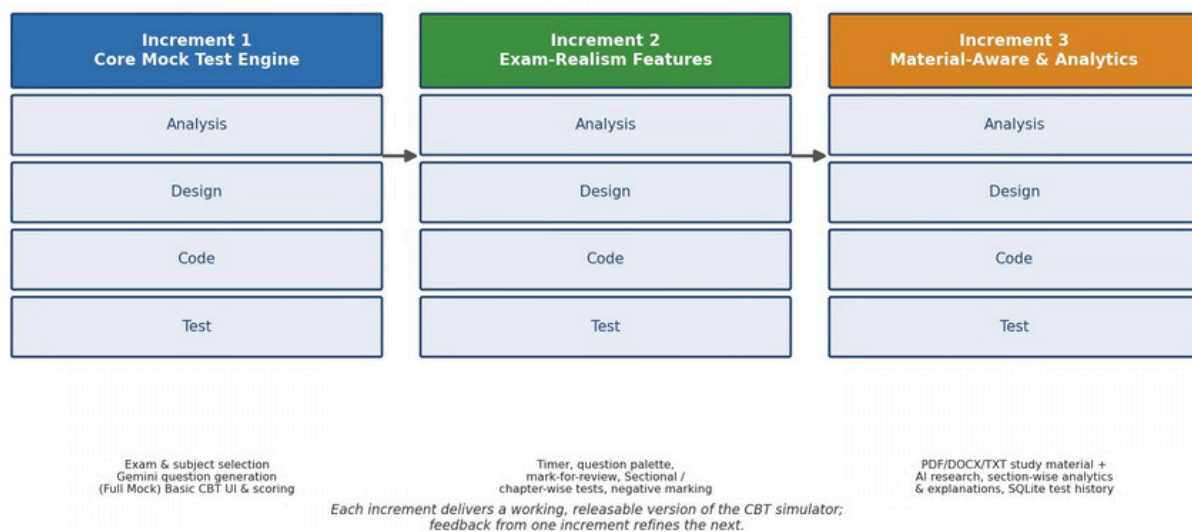


Fig. 2.2 — Incremental Model mapped to the AI Mock Test Generator project

As shown above, Increment 1 delivers the core mock-test engine (exam/subject selection, Gemini-based question generation for a Full Mock test, and a basic CBT interface with scoring) so that the AI-generation pipeline is validated as early as possible. Increment 2 adds the features that make the experience feel like a genuine CBT exam — the live timer, question palette, mark-for-review, Sectional and Chapter-wise test formats and negative marking.

Increment 3 adds the material-aware and analytics layer — PDF/DOCX/TXT study-material support, AI syllabus research, section-wise analytics with detailed explanations, and SQLite-based test history. Each increment is independently testable and demonstrable, and feedback from attempting real mock tests in one increment is used to refine the Gemini prompt design used in the next.

Advantages:

- Delivers a working version of the CBT simulator very early
- Risk associated with AI-generated question quality is isolated and addressed in the first increment
- Easier to test and debug smaller increments than one large system
- Client / user feedback after each increment can improve the next

Drawbacks:

- Requires careful upfront design of interfaces between increments (for example, the Question and ExamConfig data structures)
- Total cost can be slightly higher than a single-pass Waterfall model for a well-understood, unchanging system
- Needs disciplined planning to decide what belongs in each increment

Conclusion:

The AI Mock Test Generator can, in principle, be developed using several software process models such as Waterfall, Prototype, Incremental, Spiral, Agile and V-Model. After comparing these models against the project's characteristics — in particular its dependence on an external generative AI API whose output quality benefits from early, real-world evaluation — the Incremental Model is the most suitable. It allows the core Gemini-based question-generation pipeline to be built, demonstrated and refined first, before the exam-realism features (timer, palette, negative marking) and the material-aware analytics layer are added as later, independently testable increments. This makes it an ideal choice for a college-level AI-integrated project developed under a fixed semester timeline.

Practical No. 3

Aim:

Study Software Requirement Engineering. Student should include SRS document for current semester project.

Software Requirement Specification (SRS)

- The production of the requirements stage of the software development process is the Software Requirements Specification (SRS), also called a requirements document.
- This report lays a foundation for software engineering activities and is constructed once all requirements are elicited and analysed. The SRS is a formal report which acts as a representation of the software that enables stakeholders to review whether it meets their requirements.
- It comprises user requirements for a system as well as detailed specifications of the system's internal requirements.
- The SRS is a specification for a specific software product, program or set of applications that perform particular functions in a specific environment. First, it can be written from the perspective of the client of the system, to define the needs and expectations of the users. Second, it can be written by a developer of the system, in which case it serves as a contract document between the customer and the developer.

Introduction

The Software Requirement Specification (SRS) is a formal document that describes the complete requirements of a software system before development begins. It acts as a communication bridge between the intended users and the developer by clearly defining what the system should do and how it should perform. For the AI Mock Test Generator — CBT Exam Simulator, the SRS defines all functional and non-functional requirements needed to configure a mock test, generate questions using the Google Gemini Generative AI API, run a realistic timed CBT examination, evaluate the attempt and maintain local test history. The SRS helps developers understand user requirements, reduces development errors, improves software quality and serves as a reference throughout the software development life cycle.

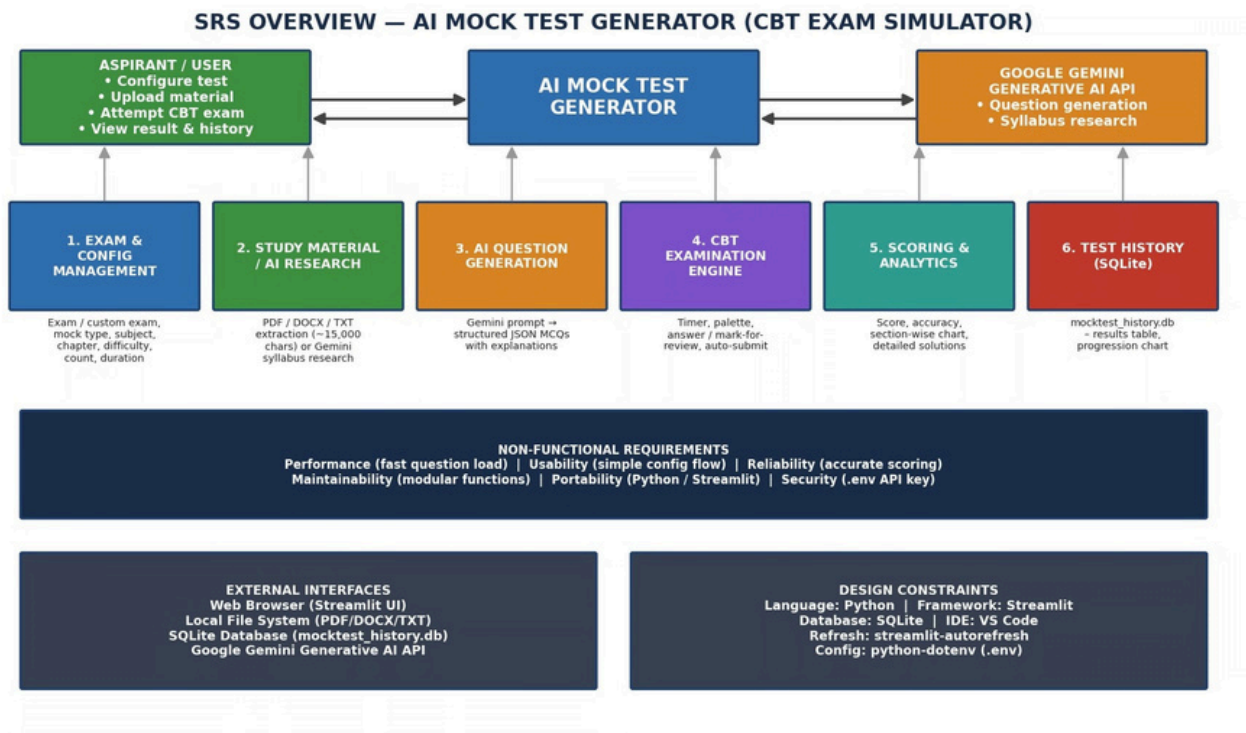


Fig. 3.1 — SRS overview diagram for the AI Mock Test Generator
SRS Should Address

a) Functionality

The AI Mock Test Generator should perform the following functions:

- Examination selection (predefined or custom)
- Mock-test type selection (Full Mock / Sectional / Chapter-wise)
- Subject and chapter/topic selection
- Study-material upload (PDF / DOCX / TXT) and text extraction
- AI-based syllabus and question-pattern research
- AI-based multiple-choice question generation
- Difficulty, question-count and duration configuration
- Negative-marking configuration
- CBT examination interface with live timer
- Question palette and navigation
- Answer selection, clear response and mark for review
- Automatic submission when time expires
- Score, accuracy and section-wise performance calculation
- Detailed, AI-generated explanations for every question
- Test-history storage and retrieval (SQLite)
- Retake-same-configuration and start-new-test functionality

b) External Interfaces

The software interacts with:

- Aspirant / User (via the Streamlit web interface)
- Google Gemini Generative AI API
- Local file system (for PDF / DOCX / TXT study-material upload)
- SQLite database (mocktest_history.db)
- Web browser (Streamlit front end)

c) Performance

The system should provide:

- Immediate response on the configuration screen
- Question generation bounded mainly by Gemini API response time
- Instant navigation, answer selection and palette updates once questions are loaded
- A live countdown timer that refreshes every second without noticeable lag
- Fast score, accuracy and section-wise analytics calculation on submission

d) Attributes

The software should have:

- Security (of the Gemini API key)
- Reliability
- Maintainability
- Portability
- Usability

e) Design Constraints

- Programming Language: Python
- Web / Application Framework: Streamlit
- Generative AI: Google Gemini Generative AI API
- Database: SQLite
- PDF Processing: pdfplumber
- DOCX Processing: python-docx
- Configuration: python-dotenv (.env, GEMINI_API_KEY)
- Timer Refresh: streamlit-autorefresh
- IDE: Visual Studio Code
- Operating System: Windows / Linux

Name of System

Enrollment No.: 2403051050469

AI Mock Test Generator — CBT Exam Simulator

Assumptions

Before using the AI Mock Test Generator:

- A valid Gemini API key is configured in the .env file as GEMINI_API_KEY.
- An internet connection is available so that the Gemini Generative AI API can be reached.
- The aspirant selects a supported examination or provides a custom examination name.
- Study material, when uploaded, is a readable PDF, DOCX or TXT file.
- The current version of the system does not require the aspirant to create an account; each session is self-contained.

Type of User

Unlike many transactional systems, the AI Mock Test Generator does not implement user registration, login, admin accounts or persistent user profiles — these are explicitly outside the current scope of the supplied implementation. There is a single user role:

Aspirant / User

The aspirant provides, at the start of a session:

- Target examination (or custom examination name)
- Mock-test type, subject(s) and chapter/topic (where applicable)
- Difficulty, question count, duration and negative-marking preference
- Optional study material (PDF / DOCX / TXT) or an AI-research request

The aspirant can:

- Configure and generate a mock test
- Attempt the CBT examination (select, clear, mark for review, navigate)
- Submit the exam manually or let the timer expire
- View the score, accuracy, section-wise analysis and detailed explanations
- Retake the same configuration or start a new test
- View historical test attempts and a score-progression chart

Requirement 1: Exam & Test Configuration

Req 1.1: Select Examination and Mock Type

Input

- Examination (SBI PO / IBPS PO / Railway RRB NTPC / SSC CGL / SSC CHSL / Custom)
- Mock-test type (Full Mock / Sectional / Chapter-wise)
- Subject(s) and, for Chapter-wise tests, a chapter/topic

Output

Configuration accepted and carried forward to the study-material step.

Processing

The system validates that a subject is selected for Sectional and Chapter-wise tests and that a chapter/topic is entered for Chapter-wise tests; otherwise an appropriate message is shown.

Req 1.2: Set Difficulty, Question Count, Duration and Negative Marking

Input

- Difficulty (Easy / Medium / Hard / Mixed)
- Number of Questions (5–100, step 5)
- Duration in minutes (5–180, step 5)
- Negative marking on/off, with a value from 0 to 2.0 (default 0.25)

Output

Final test configuration ready to be sent for question generation.

Requirement 2: Study Material / AI Research

Req 2.1: Upload Study Material

Input

One or more PDF, DOCX or TXT files.

Output

Combined extracted reference text (truncated to approximately 15,000 characters).

Processing

The system extracts text using pdfplumber (PDF) or python-docx (DOCX), or reads TXT directly, then concatenates and truncates the result before it is used as reference material for question generation.

Req 2.2: Request AI Syllabus Research

Input

Selected examination, mock type, subject(s) and chapter/topic (no file upload).

Output

A concise, Gemini-generated research summary covering syllabus, standard question pattern, topic weightage and difficulty characteristics, used in place of uploaded material.

Requirement 3: AI Question Generation

Req 3.1: Generate Questions

Input

Examination, mock type, subjects, difficulty, chapter/topic, reference material (uploaded or AI-researched) and required question count.

Output

A structured JSON list of questions, each with an ID, section, question text, four options, correct answer, explanation and difficulty tag.

Processing

The system builds a prompt from the above inputs, sends it to the Gemini Generative AI API, and parses the JSON response using Python's JSON processing. If the response is malformed or generation fails, the interface offers a Try Again option rather than proceeding with an incomplete question set.

Requirement 4: CBT Examination Interface

Req 4.1: Display Question

Output

Current question number, total number of questions, difficulty tag, the question statement, four answer options, the live timer and the navigation palette.

Req 4.2: Question Palette

Output

Visual status of every question: Not Visited, Visited but Unanswered, Answered, Marked for Review, and Answered and Marked for Review.

Enrollment No.: 2403051050469

The palette allows the aspirant to navigate directly to any question.

Requirement 5: Answer Management

Req 5.1: Save & Next

Saves the selected answer for the current question and advances to the next question.

Req 5.2: Mark for Review & Next

Saves the answer (if any), marks the question for review, and advances to the next question.

Req 5.3: Clear Response

Removes the selected answer for the current question.

Req 5.4: Save & Mark for Review

Saves the answer and marks the question for review without moving to the next question.

Requirement 6: Timer & Submission

Req 6.1: Live Countdown Timer

Output

A continuously updating countdown implemented using Python's time module, Streamlit session state and streamlit_autorefresh, refreshing the interface every second.

Req 6.2: Manual and Automatic Submission

Input

A manual Submit action (with an “Are you sure you want to submit?” confirmation, which the aspirant can confirm or cancel) or the natural expiry of the countdown timer.

Output

The examination is evaluated and the result screen is displayed; on timer expiry this happens automatically without requiring aspirant confirmation.

Requirement 7: Scoring & Result Analytics

Req 7.1: Calculate Score and Accuracy

Output

Total questions, attempted questions, correct answers, incorrect answers, unattempted questions, final score and accuracy, computed as:

- Final Score = Correct Answers – (Incorrect Answers × Negative Mark Value)

- $\text{Accuracy} = \frac{\text{Correct Answers}}{\text{Attempted Questions}} \times 100$

Req 7.2: Section-wise Performance

Output

A bar chart showing performance broken down by section / subject.

Requirement 8: Detailed Solutions

Req 8.1: View Explanation

Output

For every question: the full question text, all four options, the correct option, the aspirant's selected (incorrect) option where applicable, the AI-generated explanation, and a status of Correct, Incorrect or Unattempted.

Requirement 9: Test History

Req 9.1: Save Result

Output

A new row is inserted into the results table of the local SQLite database mocktest_history.db, containing id, exam, subject, mock_type, score, total_questions, accuracy and date_taken.

Req 9.2: View History

Output

A list of previous attempts and a score-progression chart across attempts.

Requirement 10: Retake & New Test

- Retake Same Configuration — generates a fresh set of questions using the current configuration.
- Start New Test — returns the aspirant to examination selection and clears the current questions, answers and source material.

Requirement 11: Security

- The Gemini API key is stored via an environment variable (GEMINI_API_KEY) in a .env file rather than being hard-coded in the application.
- No personal or account credentials are collected, since the system does not implement registration or login.
- Uploaded study material and generated questions remain local to the running session and the local SQLite file.

Requirement 12: Reliability

- Accurate score and accuracy calculation using the documented formulas.
- Consistent answer state during navigation within a session.
- Graceful handling of missing API keys, file-reading errors, Gemini research errors, question-generation errors and invalid JSON, with a Try Again option where appropriate.

Requirement 13: Portability

The AI Mock Test Generator can run on:

- Windows
- Linux

It is accessible using any standard web browser capable of running a local Streamlit application, for example Google Chrome, Microsoft Edge or Mozilla Firefox.

Conclusion:

The Software Requirement Specification (SRS) for the AI Mock Test Generator — CBT Exam Simulator provides a complete description of the software requirements needed to configure, generate, attempt, score and review AI-powered mock tests. It defines both functional requirements (exam and test configuration, study material / AI research, AI-based question generation, the CBT examination interface, answer management, timer and submission, scoring and analytics, detailed solutions and test history) and non-functional requirements (performance, usability, reliability, maintainability, portability and security). It also deliberately documents what is outside the current scope — user registration, login, admin accounts and a cloud/multi-user database — so that the delivered system is evaluated against what it actually implements. This document serves as a foundation for the design, development, testing and maintenance of the AI Mock Test Generator, ensuring that the final software meets aspirant requirements and provides a secure, reliable and realistic mock-examination experience.

Practical No. 4

Aim:

Develop Software Project Management Planning (SPMP) for the specified module.

1. Introduction

1.1 Purpose

This document outlines the Software Project Management Plan (SPMP) for the AI Mock Test Generator — CBT Exam Simulator. It defines the objectives, scope, project planning, scheduling, resource allocation, risk management and quality assurance required for the successful development of the system.

1.2 Scope

The project aims to develop a web-based mock-test platform that helps competitive-exam aspirants practise using AI-generated questions inside a realistic Computer-Based Test (CBT) interface. The system allows an aspirant to configure an examination, subject and difficulty, supply study material or request AI research, generate questions using the Google Gemini Generative AI API, attempt a timed exam, receive a score with section-wise analytics and explanations, and review historical attempts stored locally in SQLite.

1.3 Assumptions and Constraints

- The project is developed using Python and Streamlit.
- The system does not implement user registration, login or admin accounts; every session is self-contained for a single aspirant.
- The module focuses on core functionalities: exam configuration, study material / AI research, AI question generation, the CBT engine, scoring and test history.
- Development is carried out by a small team within the specified academic project duration.
- A valid Gemini API key and an active internet connection are required for question generation.

Project Scheduling & Tracking

Project scheduling and tracking is an activity that distributes estimated effort across the planned project duration by assigning effort to different software engineering tasks.

Scheduling Principles

- **Compartmentalization:** Divide the project into manageable modules (configuration, material processing, AI generation, CBT engine, scoring, history).
- **Interdependency:** Identify dependencies among modules — for example, question generation depends on configuration and material processing being complete.
- **Time Allocation:** Assign estimated time to every task.
- **Effort Validation:** Verify that the estimated effort, especially around Gemini API integration, is practical.
- **Define Responsibilities:** Assign work to each team member.
- **Define Outcomes:** Define expected deliverables for every phase.

Scheduling Methods

Two scheduling techniques applicable to this project are:

Program Evaluation and Review Technique (PERT)

PERT is a project management technique used to manage uncertainty in software development — particularly relevant here because the time needed to tune Gemini prompts for reliable JSON output cannot be estimated with full confidence in advance.

Key Features

- Uses probabilistic time estimates
- Focuses on project milestones
- Helps estimate overall project completion time

Time Estimates

- **Optimistic Time (O):** Minimum time required
- **Most Likely Time (M):** Expected completion time
- **Pessimistic Time (P):** Maximum possible time

Benefits

- Provides realistic project scheduling
- Helps identify activities with higher project risk, such as AI prompt tuning
- Supports better planning under uncertain conditions

Critical Path Method (CPM)

CPM is a deterministic scheduling method used for the parts of the project with well-defined activities, such as building the CBT interface and scoring logic.

Key Features

- Uses fixed activity durations
- Identifies the project's critical path
- Improves resource allocation and scheduling

Time Calculations

- Earliest Start (ES)
- Earliest Finish (EF)
- Latest Start (LS)
- Latest Finish (LF)
- Slack = LS – ES

Benefits

- Identifies bottlenecks clearly
- Improves project monitoring
- Ensures timely completion of critical tasks such as Gemini integration

2. Project Organization

2.1 Roles and Responsibilities

<i>Role</i>	<i>Responsibilities</i>
Project Manager	Oversees the project, assigns tasks, monitors progress and manages deadlines.
AI Integration Developer	Designs the Gemini prompt, integrates the Generative AI API, and parses / validates the returned JSON questions.
Streamlit / Frontend Developer	Builds the exam-configuration screens and the CBT examination interface, including the timer and question palette.
Backend / Database Developer	Implements scoring logic, session-state management and the SQLite test-history module.
Tester	Performs unit, integration and system testing of every module, including AI-response validation.
Documentation Lead	Prepares project reports, this SRS/SPMP documentation and user-facing help text.

2.2 Stakeholders

- End Users: Competitive-exam aspirants (SBI PO, IBPS PO, Railway RRB NTPC, SSC CGL, SSC CHSL and custom-exam candidates).
- Project Team: The development team responsible for building and testing the system.
- Mentors / Professors: Guide, monitor and evaluate the project as part of the Software Engineering Laboratory course.

3. Project Planning

3.1 Work Breakdown Structure (WBS)

The project planning includes:

Requirement Analysis

Identify system requirements such as:

- Exam & configuration management
- Study material / AI research
- AI question generation
- CBT examination engine
- Answer management & timer
- Scoring & analytics
- Test history

Determine software and API requirements (Gemini API quota, pdfplumber / python-docx availability).

System Design

- Design the class, sequence, activity, state and ER diagrams (Practical 7).
- Design the Gemini prompt and JSON response schema.
- Design the SQLite results table schema.
- Prepare the Streamlit UI flow for configuration and the CBT screen.

Implementation

Develop:

- Exam Configuration Module
- Study Material / AI Research Module
- AI Question Generation Module
- CBT Examination Module
- Scoring & Analytics Module

- Test History Module

Testing & Debugging

- Unit Testing
- Integration Testing
- System Testing
- Bug fixing, especially around malformed Gemini JSON responses

Deployment & Maintenance

- Deploy the Streamlit application
- Collect informal user feedback on generated-question quality
- Fix reported issues and refine the Gemini prompt
- Monitor API quota and improve performance

3.2 Human Resources

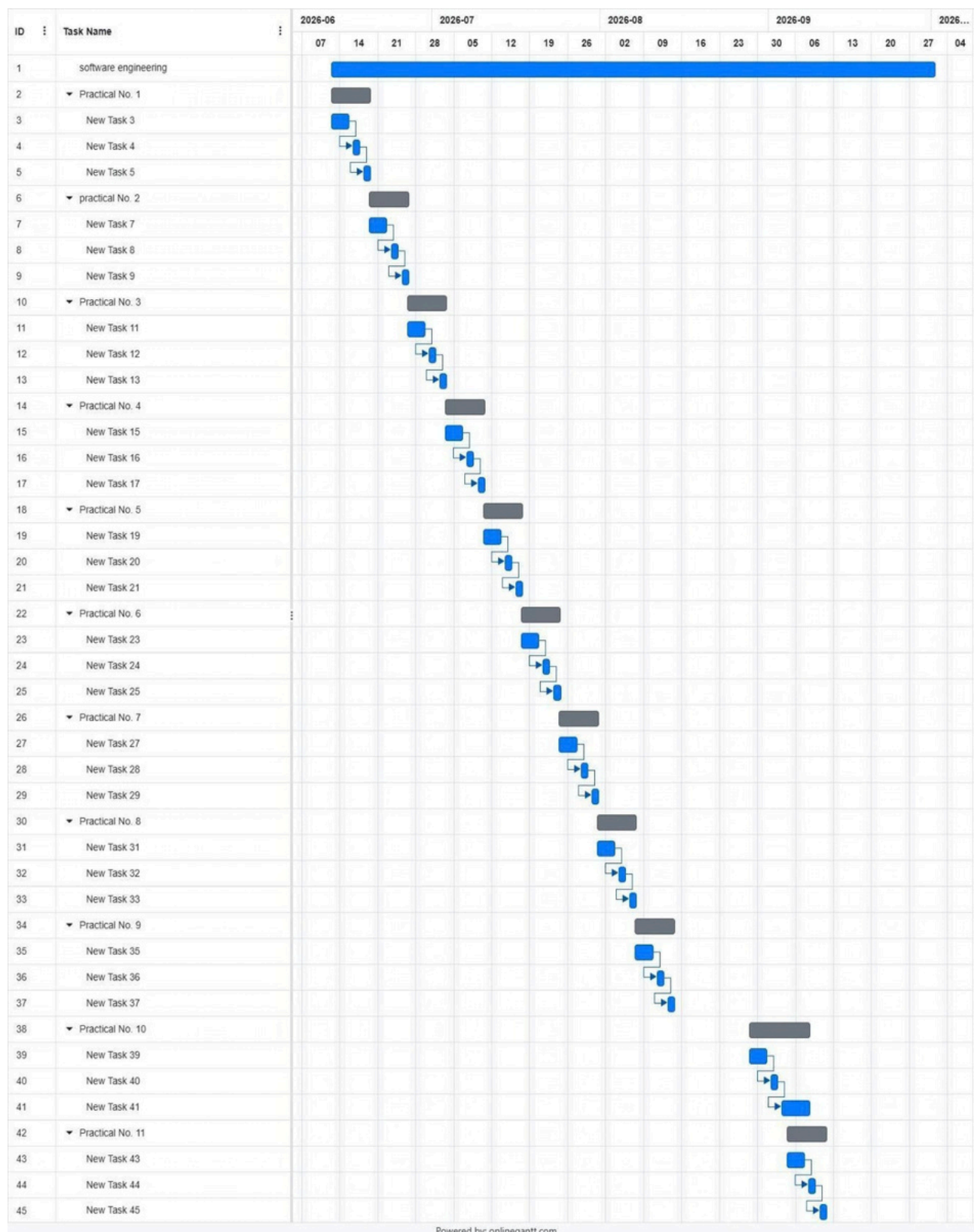
The project team consists of:

- One AI Integration Developer responsible for the Gemini prompt and question-generation pipeline.
- One Frontend / Streamlit Developer responsible for the configuration screens and CBT interface.
- One Backend Developer responsible for scoring, session-state and the SQLite history module.
- One Tester responsible for testing all modules, including AI-response validation.
- One Project Manager / Documentation Lead responsible for coordination and reporting.

Gantt Chart

A Gantt chart is used to represent the Software Engineering Laboratory activities for this project over time. It shows activity start date, activity end date, duration, task overlap, milestones and dependencies.

Gantt Chart for AI Mock Test Generator



Powered by: onlinegantt.com

4. Resource Allocation

4.1 Tools & Technologies

- Programming Language: Python
- Framework: Streamlit
- Generative AI: Google Gemini Generative AI API •

Database: SQLite

- Data Processing: Pandas
- PDF Processing: pdfplumber
- DOCX Processing: python-docx
- Configuration: python-dotenv
- Timer Refresh: streamlit-autorefresh
- Version Control: GitHub
- IDE: Visual Studio Code

5. Risk Management

<i>Risk</i>	<i>Impact</i>	<i>Mitigation Strategy</i>
Gemini API returns malformed / invalid JSON	High	Strict JSON-schema response validation, and a Try Again retry option on failure.
Gemini API quota / availability issues	High	Graceful error handling, clear user messaging, and caching / reuse of generated questions where possible.
Poor quality or inaccurate AI-generated questions	Medium	Iterative prompt refinement using feedback from early increments; display AI-generated explanations for transparency.
Study-material extraction errors (corrupt PDF/DOCX)	Medium	Defensive file-reading with clear error messages and fallback to AI research.
Time constraints within the academic semester	High	Follow the Incremental Model with weekly milestones (Practical 1–11).
Timer / session-state inconsistency in	Medium	Careful session-state design and testing of navigation, refresh and auto-submit

Enrollment No.: 2403051050469

<i>Risk</i>	<i>Impact</i>	<i>Mitigation Strategy</i>
Streamlit		behaviour.

6. Quality Assurance

6.1 Testing Strategy

- Unit Testing: Test configuration validation, material extraction, Gemini prompt construction / JSON parsing, timer logic and scoring formulas individually.
- Integration Testing: Verify the complete flow from configuration through question generation, exam attempt, scoring and history saving.
- System Testing: Validate the complete AI Mock Test Generator across multiple examinations, mock types and difficulty levels.
- User Acceptance Testing: Obtain feedback from fellow students attempting full mock tests before final submission.

6.2 Performance Metrics

The system's performance will be evaluated using:

- Question-generation success rate (valid JSON returned by Gemini)
- CBT navigation and timer responsiveness
- Scoring and accuracy calculation correctness
- Test-history save / retrieval reliability
- Overall aspirant (user) satisfaction with the mock-test experience

7. Conclusion

This Software Project Management Plan (SPMP) provides a structured approach for developing the AI Mock Test Generator — CBT Exam Simulator. By following systematic project planning, an Incremental development approach with clear scheduling, appropriate resource allocation, proactive risk management around the Gemini AI integration, and disciplined quality assurance practices, the development team can deliver a reliable, realistic and useful mock-test platform for competitive-exam aspirants.

Practical No. 5

Aim:

Cost and Effort Estimation using different Software Cost Estimation models.

Objectives :

- To make use of the COCOMO (Constructive Cost Model) for estimating the cost of developing the AIMockTestGenerator — CBT Exam Simulator.
- To analyse the effort, development time, productivity and staffing requirements for the project.
- To classify the AIMockTest Generator project as Organic, Semi-detached or Embedded and select the appropriate COCOMO model parameters for accurate cost estimation.

Theory:

Introduction to Software Cost Estimation

Software cost estimation is a critical process in Software Engineering used to predict the effort, time, manpower and cost required to develop a software system. Accurate cost estimation helps in planning, resource allocation and budgeting. One of the most widely used models for software estimation is the COCOMO Model, proposed by Barry Boehm.

COCOMO Model (Constructive Cost Estimation Model)

The COCOMO model provides a mathematical formula for estimating:

- Effort (PM – Person-Months)
- Development Time (T_{dev} – in months)
- Average Staff Size
- Productivity
- Cost (Budget in ₹)

The model works in three stages:

1. Basic COCOMO — rough estimate of effort and development time.
2. Intermediate COCOMO — adds cost drivers (product, hardware, personnel, project attributes).
3. Detailed (Complete) COCOMO — adds detailed, phase-sensitive analysis of each phase of development (design, coding, testing).

Software Development Project Classification

<i>Model</i>	<i>Project Size</i>	<i>Nature of Project</i>	<i>Innovation</i>	<i>Deadline</i>	<i>Environment</i>
Organic	Typically 2–50 KLOC	Small, well-understood application, experienced developers.	Little	Not tight	Familiar & in-house
Semi-detached	Typically 50–300 KLOC	Medium size, mixed experience, some unfamiliar sub-systems (e.g. external API integration).	Medium	Medium	Medium
Embedded	Typically 300+ KLOC	Large, real-time, complex interfaces, tight hardware coupling.	Significant	Tight	Complex hardware & interfaces

Classifying the AI Mock Test Generator

The AI Mock Test Generator is classified as a Semi-detached project. Although the overall application (a Streamlit CRUD-style interface with a local SQLite database) is well understood, the project also integrates the Google Gemini Generative AI API for structured question generation, manages a live countdown timer through Streamlit session state and streamlit-autorefresh, and extracts text from multiple document formats — areas in which the development team has only partial prior experience. This mixture of familiar and unfamiliar sub-systems, together with a small-to-medium codebase and a moderate project deadline, matches the Semi-detached category rather than a purely Organic or a large, hardware-coupled Embedded project.

Formulas Used in Basic COCOMO

1. Effort (Person-Months, PM):

$$\text{Effort (E)} = a \times (\text{KLOC})^b$$

2. Development Time (Months):

$$T(\text{dev}) = c \times (\text{Effort})^d$$

3. Average Staff Size (Persons):

$$\text{Staff Size} = \text{Effort} / T(\text{dev})$$

4. Productivity:

$$\text{Productivity} = \text{KLOC} / \text{Effort}$$

Cost (₹):

$$\text{Cost} = \text{Staff Size} \times T(\text{dev}) \times \text{Salary per Person}$$

<i>Project</i>	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>
Organic	2.4	1.05	2.5	0.38
Semi-detached	3.0	1.12	2.5	0.35
Embedded	3.6	1.20	2.5	0.32

Every line of source text is counted as one LOC irrespective of the actual number of instructions on that line; if a single instruction spans several lines (say n lines), it is considered to be n LOC. The constants a, b, c, d above for the Organic, Semi-detached and Embedded categories are those given by Boehm.

Problem:

Assume that the AI Mock Test Generator software project is estimated to contain 8,000 lines of code (8 KLOC), covering the configuration screens, the Gemini prompt / JSON-parsing pipeline, the CBT examination engine, scoring logic and the SQLite test-history module. The average monthly salary of a software engineer is ₹11,500. Using the Basic COCOMO Model for a Semi-detached project, estimate the following:

1. Effort (Person-Months)
2. Development Time (Months)
3. Average Staff Size

4. Productivity
5. Total Development Cost

Solution (Basic COCOMO Model — AI Mock Test Generator)

Given:

- Project: AI Mock Test Generator — CBT Exam Simulator
- Project Type: Semi-detached
- KLOC = 8
- Salary per Software Engineer = ₹11,500 / month

COCOMO Formula (Semi-detached):

- Effort (E) = $3.0 \times (\text{KLOC})^{1.12}$
- Development Time (Tdev) = $2.5 \times (\text{E})^{0.35}$
- Staff Size = E / Tdev
- Productivity = KLOC / E
- Cost = $\text{Staff Size} \times \text{Tdev} \times \text{Salary}$

Step 1: Effort Estimation

- $\text{E} = 3.0 \times (8)^{1.12}$
- $\text{E} \approx 30.8 \text{ Person-Months} \approx 31 \text{ Person-Months}$

Step 2: Development Time

- $\text{T(dev)} = 2.5 \times (31)^{0.35}$
- $\text{T(dev)} \approx 8.3 \text{ Months} \approx 8 \text{ Months}$

Step 3: Average Staff Size

- $\text{Staff Size} = 31 / 8.3$
- $\approx 4 \text{ Persons}$

Step 4: Cost Estimation

- $\text{Cost} = 4 \times 8 \times 11,500$
- $= ₹3,68,000$

Step 5: Productivity

- $\text{Productivity} = 8 / 31$
- $\approx 0.258 \text{ KLOC} / \text{Person-Month}$

Conclusion

The AI Mock Test Generator is estimated to require approximately 31 Person-Months of effort with a development time of about 8 months. The project will need an average team of 4 software engineers, and the estimated development cost is ₹3,68,000. These estimates smaller than a typical multi-module transactional system, reflecting the project's compact, modular Streamlit codebase — help in effective project planning, budgeting and resource allocation for the AI Mock Test Generator.

Practical No. 6

Aim:

To prepare a system analysis and design for the AI Mock Test Generator — CBT Exam Simulator using structured design techniques such as Data Flow Diagrams (DFD) and Data Dictionary.

Objective:

The objective of this experiment is to define the scope and boundaries of the system, represent data flow between different components using DFDs, document system data using a data dictionary, and ensure clear communication between stakeholders and developers. A DFD may be used as a communication tool between a system analyst and anyone involved in the project, and acts as a starting point for redesigning a system. A DFD is also called a data-flow graph or bubble chart.

Apparatus/Components: Word, PowerPoint, Microsoft Visio, Edraw Max, Python (matplotlib) for diagram preparation.

Theory:

A Data Flow Diagram (DFD) is a graphical representation of data movement within a system. It consists of external entities, processes, data stores and data flows. A Context Level DFD provides an overview of the entire system as a single process. A Data Dictionary defines the structure of data elements and their relationships. DFDs help visualise how data moves through different parts of the system, making it easier to identify inefficiencies and areas for optimisation.

- **External Entities:** A producer or consumer of data — for the AI Mock Test Generator these are the Aspirant, the Google Gemini AI API, the Local File System and the SQLite Database. Data must always originate somewhere and must always be sent to something.

- **Process:** A data transformation that changes input to output, for example “generate questions” or “calculate score”. Data must always be processed in some way to achieve a system function.

- **Data Flow:** Data moving through the system, beginning as input and transformed into output, shown as a labelled arrow.
- **Data Store:** A repository of data at rest, such as the session-held question list or the persisted SQLite results table.

Data Flow Diagramming: Guideline

- Aspirant configuration → System → Generated questions
- Study material / research request → Gemini AI API → Structured JSON questions
- Aspirant answers → System → Score, accuracy & result

The Context Level DFD shows the overall system as a single process with input / output data flow to and from external entities.

3. System Analysis

3.1 Context Level DFD (Level 0)

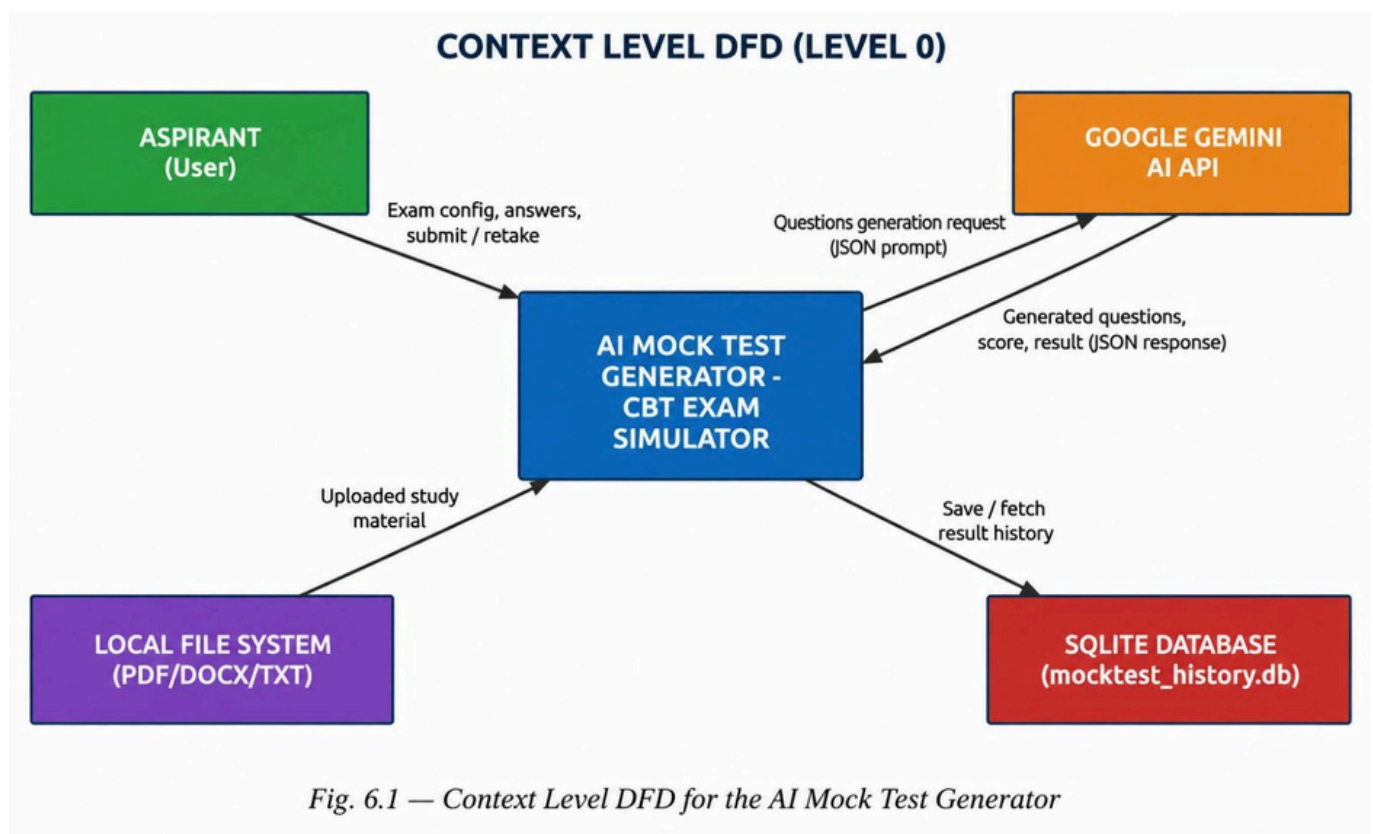


Fig. 6.1 — Context Level DFD for the AI Mock Test Generator

External Entities:

- Aspirant (User)
- Google Gemini Generative AI API
- Local File System (PDF / DOCX / TXT study material)
- SQLite Database (mocktest_history.db)

Steps used to build the DFD set

1. Create a list of activities.
2. Construct the Context Level DFD (identifies sources and sinks).
3. Construct the Level 0 DFD (identifies manageable sub-processes).
4. Construct the Level 1 DFD (identifies actual data flows and data stores).

1. List of Activities

- Configure exam, mock type, subject, difficulty, count, duration and negative marking
- Upload study material or request AI syllabus research
- Generate questions via the Gemini Generative AI API
- Attempt the CBT exam (select / clear / mark for review / navigate)
- Run the countdown timer and accept manual or automatic submission
- Evaluate answers and calculate score & accuracy
- Display section-wise analytics and detailed explanations
- Save and retrieve test history

2. Level 0 DFD (Process Breakdown)

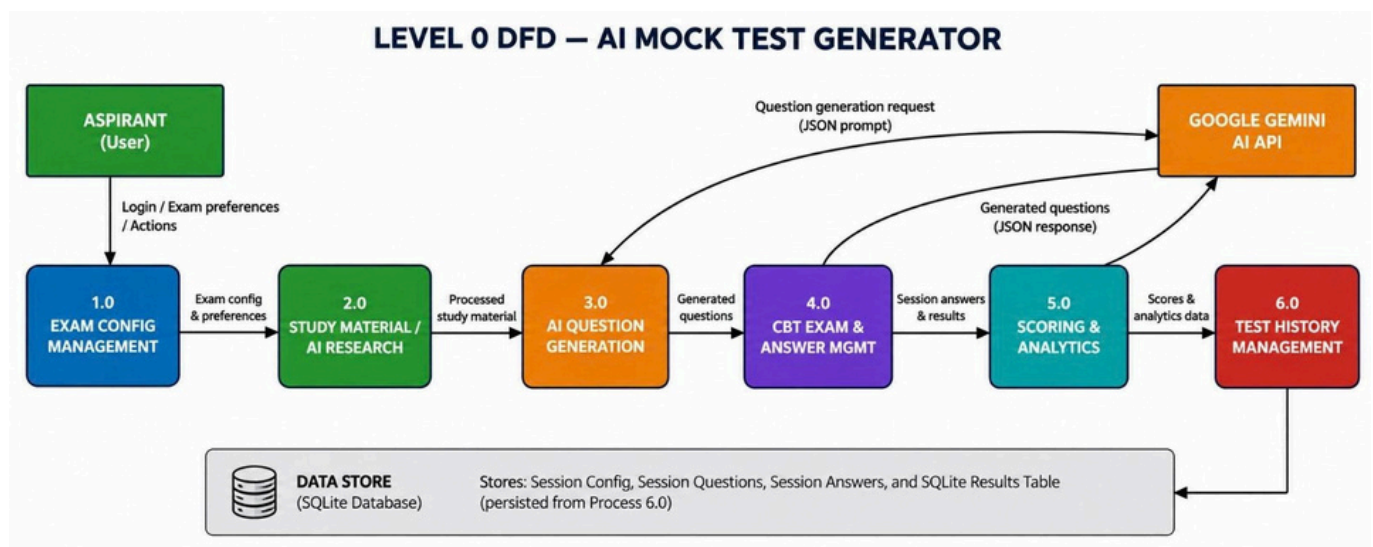


Fig. 6.2 — Level 0 DFD showing the six major processes of the AI Mock Test Generator

The overall system is decomposed into six manageable sub-processes: 1.0 Exam Configuration Management, 2.0 Study Material / AI Research, 3.0 AI Question Generation, 4.0 CBT Exam & Answer Management, 5.0 Scoring & Analytics, and 6.0 Test History Management. Process 3.0 exchanges a generation request and reference material with the Gemini AI API and receives structured JSON questions in return.

3. Level 1 DFD (Detailed Data Flows and Data Stores)

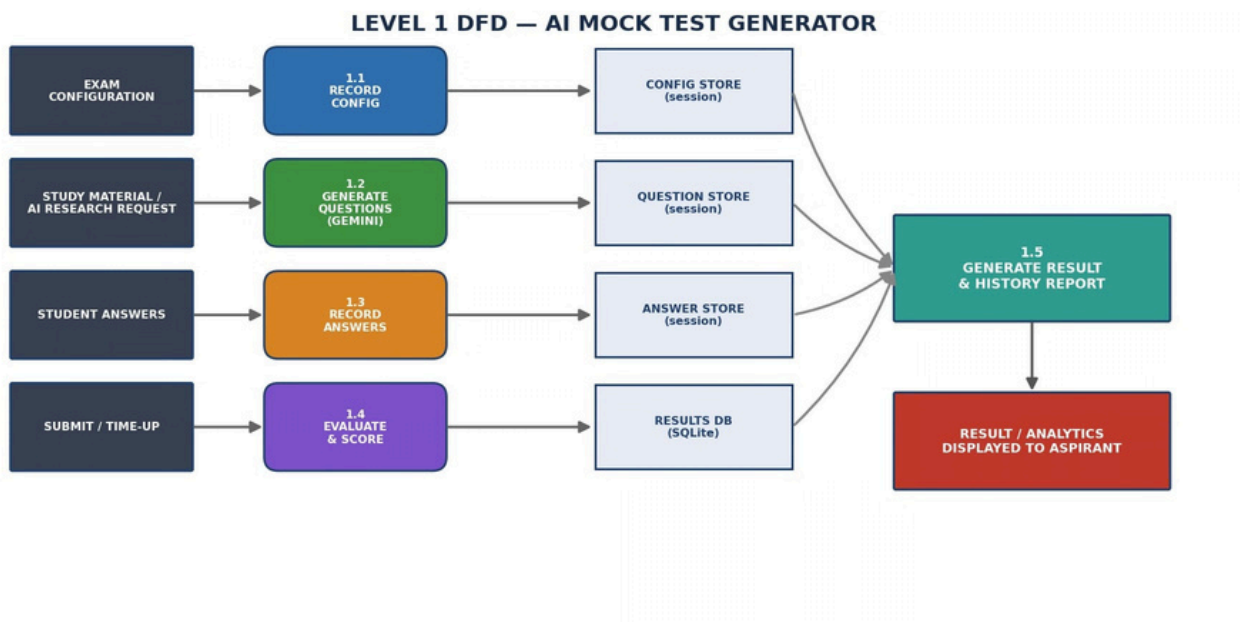


Fig. 6.3 — Level 1 DFD showing detailed processes and data stores

At Level 1, the configuration, question-generation, answer-recording and evaluation processes are shown individually, each writing to its own data store (Config Store, Question Store and Answer Store, all held in Streamlit session state) before Process 1.5 combines them to generate the result and persist a summary row to the Results DB (SQLite), which is then displayed back to the aspirant as result and analytics.

Data Dictionary:

- A data dictionary contains metadata, i.e. data about the data used by the system.

- It is important because it records what is held by the system, how it is structured, and where each piece of data lives (in-memory session state versus the persisted SQLite database).
- Unlike a traditional multi-table relational system, the AI Mock Test Generator persists only one table (results) to SQLite; the remaining structures exist only in Streamlit session state for the duration of a single run.

Structure	Field Name		Description
TestConfig (session state)	exam	String	Selected examination or custom exam name.
	mock_type	String	Full Mock / Sectional / Chapter-wise.
	subjects	List<String>	Selected subject(s) for the test.
	chapter_topic	String	Chapter/topic for Chapter-wise tests.
	difficulty	String	Easy / Medium / Hard / Mixed.
	num_questions	Integer	Number of questions (5–100).
	duration_min	Integer	Exam duration in minutes (5–180).
	negative_marking	Boolean	Whether negative marking is enabled.
	negative_value	Float	Penalty per incorrect answer (0–2.0, default 0.25).
	file_type	String	PDF / DOCX / TXT.
StudyMaterial (session state)	extracted_text	String	Extracted reference text ($\leq \sim 15,000$ characters).
	id	Integer	Unique identifier for the question.
	section	String	Subject / section the question belongs to.
	question_text	String	The MCQ statement.
	option_a..d	String	The four answer options.
	correct_answer	String	The correct option.
	explanation	String	AI-generated explanation of the answer.
Question (session state)	difficulty	String	Difficulty tag of the individual question.
	selected_option	String	The option chosen by the aspirant.
	visited	Boolean	Whether the question has been visited.
	marked_review	Boolean	Whether the question is marked for review.

Structure	Field Name	Data Type	Description
Result (PERSISTED SQLite) –	id	INT (Primary Key)	Unique identifier for each attempt.
	exam	VARCHAR	Examination attempted.
	subject	VARCHAR	Subject(s) attempted.
	mock_type	VARCHAR	Full Mock / Sectional / Chapter-wise.
	score	DECIMAL	Final calculated score.
	total_questions	INT	Number of questions in the attempt.
	accuracy	DECIMAL	Percentage accuracy.
	date_taken	DATETIME	Date and time the test was taken.

5. Procedure

1. Identify system entities, processes and data flows.
2. Construct a Context Level DFD (Level 0).
3. Develop a Level 0 process-breakdown DFD and a Level 1 DFD, detailing data stores.
4. Define the Data Dictionary, outlining both session-state and persisted (SQLite) data structures.
5. Validate the system design against the SRS from Practical 3 to ensure clarity and completeness.

6. Conclusion

This experiment successfully demonstrated how to use structured design techniques for system analysis on the AI Mock Test Generator. The DFDs provided a clear visualisation of data movement — from aspirant configuration, through Gemini-based question generation, into the CBT examination engine, and finally into scoring and locally persisted test history. The Data Dictionary ensured well-defined documentation of both transient (session-state) and persisted (SQLite) system data. Together, this structured analysis supports efficient system implementation, easier troubleshooting and improved communication about the AI Mock Test Generator's design.

Practical No. 7

Aim:

To design the AI Mock Test Generator — CBT Exam Simulator using an Object-Oriented Approach (OOA) by creating Class Diagrams, a State Diagram, Sequence Diagrams, Activity Diagrams and an ER Diagram.

Theory:

The AI Mock Test Generator allows an aspirant to configure a mock test, generate questions using the Google Gemini Generative AI API, attempt those questions inside a timed CBT interface, and receive a score with analytics and history. Object-Oriented Design (OOD) helps structure this system using objects that interact with each other. It includes:

- Class Diagrams—defining objects, attributes, methods and relationships.
- State Diagrams—describing how an object (here, the exam session) transitions between states.
- Sequence Diagrams—capturing the sequence of operations over time.
- Activity Diagrams—representing the workflow of operations.
- ER Diagrams —representing the data entities used by the system, both transient (session) and persisted (SQLite).

1. Class Diagram

The class diagram for the AI Mock Test Generator describes the structure of the system, including the classes, attributes, operations and relationships between objects. The main classes of the system are:

- ExamConfig: Represents the aspirant's chosen exam, mock type, subjects, difficulty, question count, duration and negative-marking preference.
- StudyMaterial: Represents uploaded reference material and its extracted text.
- AIQuestionGenerator: Builds the Gemini prompt, calls the Generative AI API and produces Question objects (or a syllabus-research summary).
- Question: Represents a single AI-generated MCQ with its options, correct answer, explanation and difficulty.
- ExamSession: Manages the live attempt — the current question index, the timer start time, and the aspirant's in-progress answers.

- **Answer:** Represents the aspirant's response to a single question, including its visited and marked-for-review state.
- **ScoringEngine:** Evaluates an ExamSession and produces the score, accuracy and section-wise breakdown.
- **ResultRecord:** Represents a persisted summary of a completed attempt, saved to and retrieved from the SQLite results table.
- **Association** — a relationship between instances of two classes.
- **Aggregation** — an association in which one class belongs to a collection, shown with a line from the whole to its parts.
- **Dependency** — shown where one class uses another to perform its behaviour (for example, AIQuestionGenerator uses StudyMaterial).

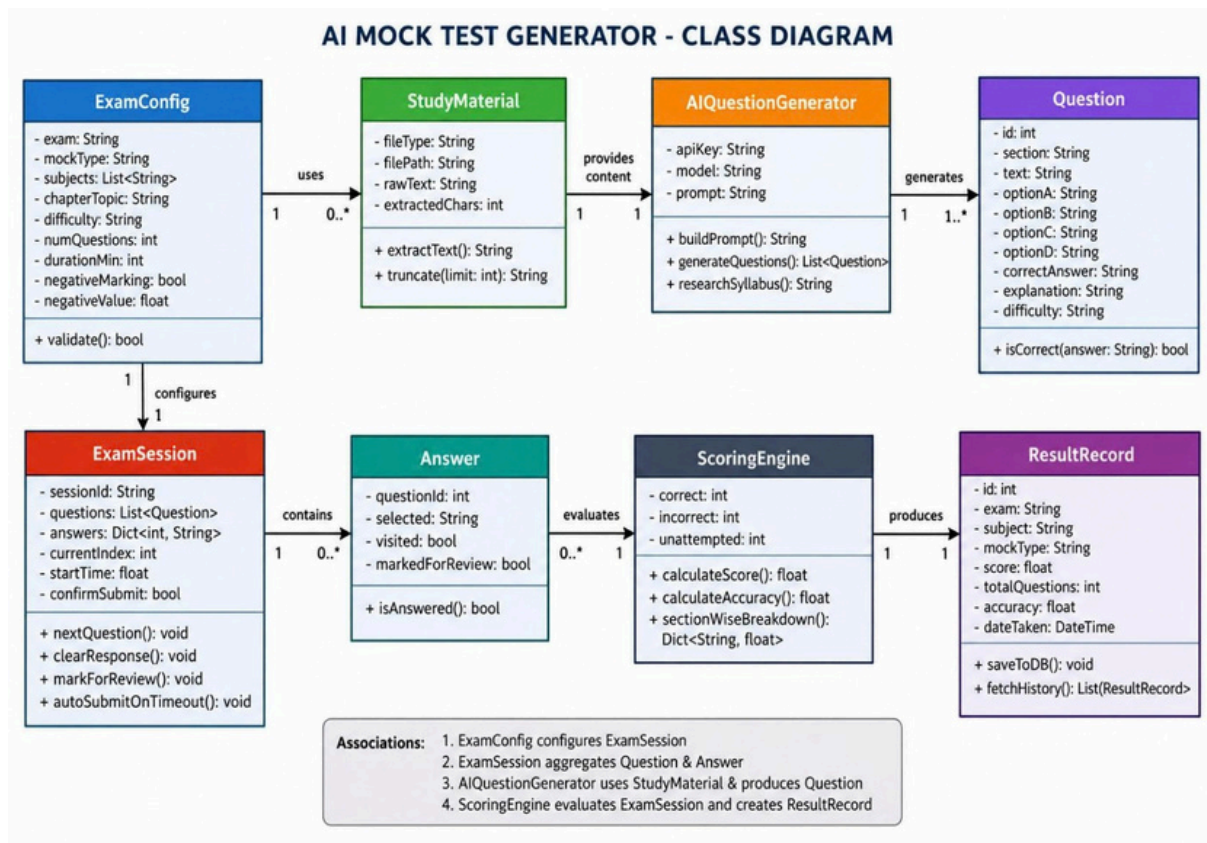


Fig. 7.1 — Class Diagram for the AI Mock Test Generator

2. State Diagram

The state diagram below describes the lifecycle of a single ExamSession object, from the moment the aspirant begins configuring a test to the point at which the result is saved to history.



Fig. 7.2 — State Diagram for the Exam Session lifecycle

The session begins in the Config state, moves to Generating Questions once the aspirant requests a test, and returns to Config if generation fails so that the aspirant can retry. On success it moves to In Progress, where the aspirant attempts questions until the exam is Submitted (manually or by timer expiry). The session is then Evaluated and the result is shown, before finally being Saved to History in the local SQLite database.

3. Sequence Diagram

This is the sequence diagram for the AI Mock Test Generator, showing how the Aspirant, the Streamlit UI, ExamConfig, AIQuestionGenerator, the Gemini API, ExamSession, ScoringEngine and the SQLite database interact over the course of a single mock-test attempt. The aspirant first configures the exam, the UI requests question generation from Gemini through AIQuestionGenerator, the generated questions are loaded into an ExamSession, the aspirant attempts and submits the exam, ScoringEngine evaluates the answers, and finally the ResultRecord is saved to SQLite and the result is displayed back to the aspirant.

The main objects involved in this sequence are:

- Aspirant
- Streamlit UI
- ExamConfig

- AIQuestionGenerator
- Gemini API
- ExamSession
- ScoringEngine
- SQLite DB

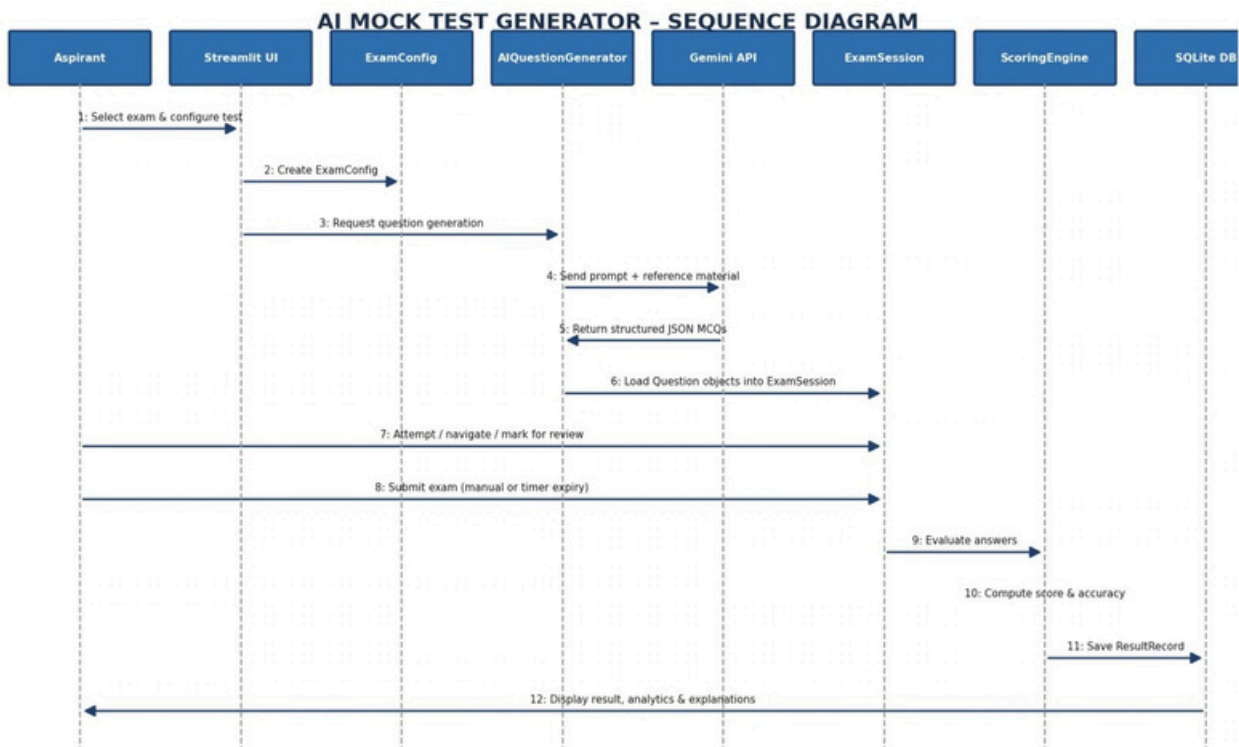


Fig. 7.3 — Sequence Diagram for the AI Mock Test Generator

4. Activity Diagram

This is the activity diagram for the AI Mock Test Generator, showing the flow between the activities of configuring a test, generating questions, attempting the CBT exam, and reviewing the result. The main activities involved are:

- Exam-configuration activity
- Study-material / AI-research activity
- AI question-generation activity (with a retry loop on failure)
- CBT-attempt activity (select / mark / clear / navigate)
- Scoring and result-display activity
- History-saving and retake / new-test activity

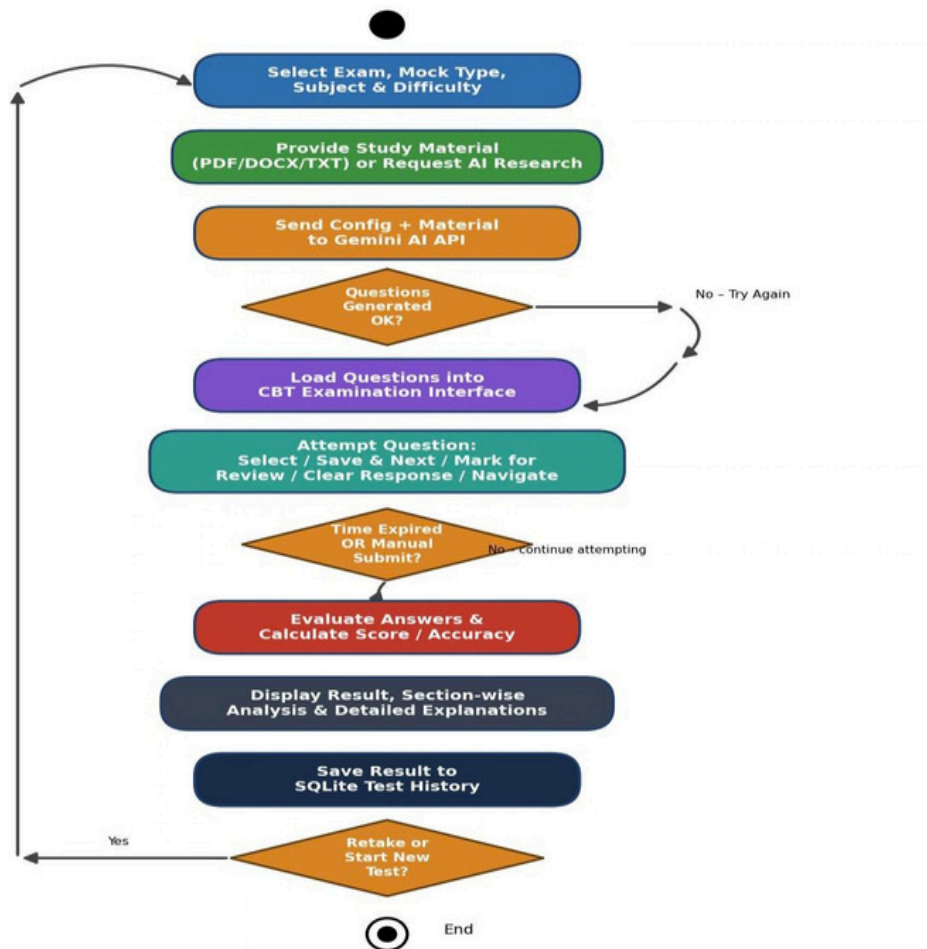


Fig. 7.4 — Activity Diagram for the AI Mock Test Generator

Features of the Activity Diagram:

- The aspirant configures the exam and either uploads study material or requests AI research before question generation begins.
- If Gemini question generation fails, the flow loops back so the aspirant can try again rather than the session failing outright.
- Once questions are generated, the aspirant can freely select, save, clear or mark-for-review any question via the palette, until either the timer expires or the aspirant manually submits.
 - After evaluation, the aspirant may retake the same configuration (looping back to question generation) or start an entirely new test (looping back to exam configuration).

5. ER Diagram

The ER (Entity-Relationship) Diagram represents the data model of the AI Mock Test Generator. It is used to structure data and to define the relationship between the data groups used by the system's functionality. Unlike a fully relational, multi-table system, only one entity — Result — is actually persisted to the local SQLite database; TestConfig, StudyMaterial, Question and Answer exist only in Streamlit session state for the duration of a single run, and are shown here to make the complete data model explicit.

AI Mock Test Generator entities and attributes are:

1. TestConfig Entity (session-state): config_id, exam, mock_type, subjects, chapter_topic, difficulty, num_questions, duration_min, negative_marking, negative_value.
2. StudyMaterial Entity (session-state): material_id, config_id (FK), file_type, extracted_text.
3. Question Entity (session-state): question_id, config_id (FK), section, question_text, option_a..d, correct_answer, explanation, difficulty.
4. Answer Entity (session-state): answer_id, question_id (FK), selected_option, visited, marked_review.
5. Result Entity (PERSISTED – SQLite results table): id, exam, subject, mock_type, score, total_questions, accuracy, date_taken.

ER DIAGRAM — AI MOCK TEST GENERATOR (SESSION & PERSISTED DATA)

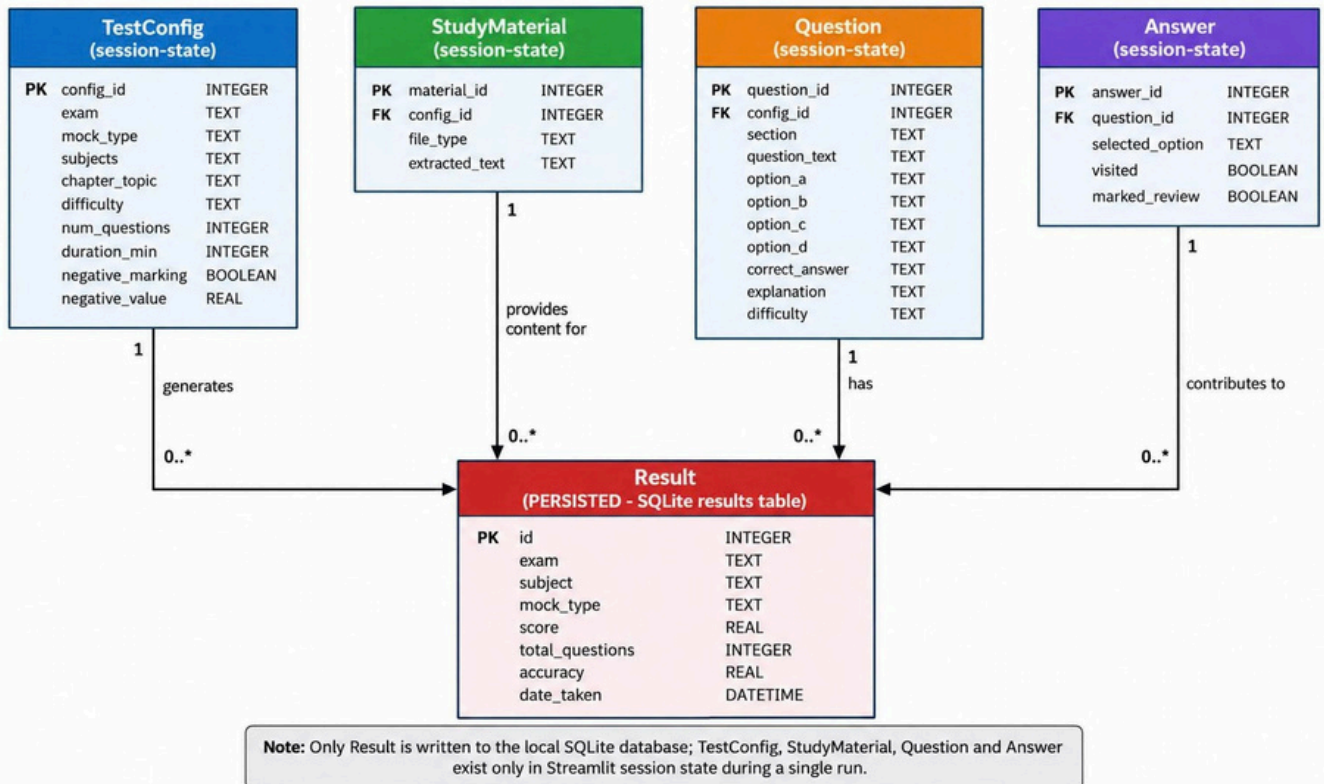


Fig. 7.5 — ER Diagram for the AI Mock Test Generator

Fig. 7.5 — ER Diagram for the AI Mock Test Generator

Conclusion:

The design of the AI Mock Test Generator — CBT Exam Simulator using the Object-Oriented Approach (OOA) provides a structured and scalable framework for developing a realistic, AI-driven mock-examination system. By employing UML diagrams such as the Class Diagram, State Diagram, Sequence Diagram, Activity Diagram and an ER Diagram, the system has been modelled to ensure clarity in functionalities, relationships and interactions — in particular, the interaction between the application and the external Google Gemini Generative AI API. This modular design simplifies understanding of the system and enhances its maintainability and future scalability, for example if user accounts or a cloud database are added in a later version. The use of OOA ensures that aspirant requirements are effectively translated into system behaviour, enabling a smooth, realistic mock-test experience from configuration through to detailed result analytics.

Practical No. 8

Aim: - Defining Coding Standards and walk through.

Introduction

Coding standards play a crucial role in ensuring the maintainability, readability, and scalability of software projects. A well-defined coding standard helps developers write consistent, high quality code, reducing technical debt and improving collaboration. A code walkthrough process is an essential practice that enables developers to review, identify issues, and optimize code for efficiency and security. This document outlines the importance of coding standards, key best practices, and a structured code walkthrough process tailored for the platform. Objectives of Coding Standards The primary objectives of defining and adhering to coding standards include improving code readability, ensuring that the code is structured and formatted consistently for easier understanding and modification. It enhances maintainability by ensuring that properly documented and structured code simplifies debugging and future enhancements. Coding standards reduce bugs by minimizing common coding errors and enhancing reliability. Additionally, they ensure security and performance by helping prevent vulnerabilities and optimizing execution efficiency. Lastly, they facilitate team collaboration by enforcing a uniform coding style that ensures seamless teamwork among developers.

Key Coding Standards :

- ***Code Formatting :***

Consistent indentation should be used to enhance readability and maintainability. Standard naming conventions should be followed to ensure clarity and consistency across the codebase. Line lengths should be kept within recommended limits to improve code presentation and readability.

- ***Documentation***

Proper documentation is critical for code clarity. Developers should maintain clear and concise comments that describe the purpose and functionality of the code. All functions and modules should include relevant documentation to facilitate easy understanding and maintenance. An up-to-date README file and API documentation should be maintained to assist developers in understanding the project's structure and functionality.

- **Error Handling and Logging**

Structured exception handling should be implemented to handle potential errors gracefully. Logging frameworks should be used to track and record errors efficiently rather than relying on ad-hoc debugging methods. Proper logging ensures that issues can be diagnosed and resolved effectively, enhancing the overall stability and reliability of the system.

- **Security Best Practices**

All user inputs should be validated and sanitized to prevent potential security threats. Passwords must be stored securely using encryption and hashing techniques. Developers should follow best practices to prevent common vulnerabilities such as SQL injection and cross-site scripting (XSS). Sensitive credentials should never be hardcoded in the source code but stored securely using environment variables or other secure storage mechanisms.

- **Version Control and Code Reviews** Version control should be managed using a structured system such as Git, ensuring all changes are properly documented and trackable. Branching strategies should be followed to maintain an organized development process. Code reviews should be a mandatory practice to identify potential issues and ensure adherence to coding standards. Reviewers should provide constructive feedback, and necessary improvements should be made before code is merged into the main branch. A code walkthrough is a structured session where developers present their code to peers for review, enhancing code quality and knowledge-sharing among the team.

- **Preparation Phase** The code module for review should be selected, and reviewers with relevant expertise should be assigned. Documentation and test cases should be prepared to support the review process.

- **Review Session** During the review session, the developer should explain the code's functionality and logic. Reviewers should check for adherence to coding standards and identify areas for optimization and possible bugs.

Feedback and Iteration Issues and suggested improvements should be documented systematically. Necessary changes should be implemented based on feedback, and follow-up reviews should be conducted if required to ensure all concerns are addressed.

□ **Tools and Best Practices**

Recommended Tools

Developers should use appropriate tools for quality assurance, such as linters and formatters to enforce coding standards automatically. Static code analysis tools help identify vulnerabilities, while CI/CD pipelines ensure automated testing and deployment to maintain code quality throughout the development cycle.

● ***Best Practices for Effective Walkthroughs***

Each session should focus on a specific code segment to ensure an efficient review process. Reviewers should provide constructive feedback, and an action item list should be maintained for post-review improvements. Automated tools should be leveraged.

```
app.py > ...
105 # -----
106 def extract_text_from_files(uploaded_files):
107     """Extract and concatenate text from PDF, DOCX, and TXT files."""
108     combined_text = ""
109     for file in uploaded_files:
110         file_type = file.name.split(".")[-1].lower()
111         try:
112             if file_type == "pdf":
113                 with pdfplumber.open(file) as pdf:
114                     for page in pdf.pages:
115                         t = page.extract_text()
116                         if t:
117                             combined_text += t + "\n"
118             elif file_type == "docx":
119                 doc = docx.Document(file)
120                 combined_text += "\n".join([p.text for p in doc.paragraphs])
```

```
app.py > ...
127
128 # NOTE: Model name updated from 'gemini-2.0-flash' to 'gemini-2.5-flash'
129 # If you hit another 429/quota error with this model, check
130 # https://aistudio.google.com for which model currently shows a
131 # non-zero free quota for your account, and swap the name here
132 # and in generate_questions() below to match.
133 GEMINI_MODEL_NAME = 'gemini-flash-latest'
134
135 def ai_research_syllabus(exam, mock_type, subjects, chapter):
136     """Use Gemini API to research syllabus and question trends."""
137     if not API_KEY:
138         st.error("GEMINI_API_KEY is not set. Please check your .env file")
139         return ""
140     genai.configure(api_key=API_KEY)
141     model = genai.GenerativeModel(GEMINI_MODEL_NAME)
```

```
app.py > ...
161
162 def generate_questions(config, source_text):
163     """Generate structured JSON exam questions using Gemini API."""
164     if not API_KEY:
165         raise ValueError("GEMINI_API_KEY is missing!")
166
167     genai.configure(api_key=API_KEY)
168     model = genai.GenerativeModel(GEMINI_MODEL_NAME)
169
170     subj_str = ", ".join(config['subjects']) if isinstance(config['subj
171     chapter_str = f" Focus specifically on Chapter/Topic: {config['chap
172
173     prompt = f"""
174     You are an expert question paper setter for Indian competitive exam
175     Generate EXACTLY {config['num_questions']} multiple-choice question
176     - Exam: {config['exam']}"""
```

```
app.py > ...
162 def generate_questions(config, source_text):
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
```

conclusion

Defining and enforcing coding standards is vital for the success of the Peer- to-Peer Lodging Exchange Platform. A structured code walkthrough process ensures quality, security, and efficiency in development. By following best practices and using tools, the team can create a maintainable, scalable, and high-performing system.

Practical No. 9

Aim:-Writethetestcasesfortheidentified module .

Introduction

TheAIMockTestGeneratoris designed to streamline the creation, configuration, computer-basedmockexaminations for aspirants. The system supports exam configurationquestiongeneration, study-material processing, timed CBT examination, scoring,analyticsToensurereliability, usability, and correctness, a systematic testing approach is essential.

Writingtestcaseshelpsvalidate that the system meets functional requirements performs operations correctly, and provides a user-friendly examination experience.

Thetestcaseswillfocusoncore modules like exam configuration, study material CBTexamination,scoringandanalytics, timer handling, and test history, ensuring thesystemperformsreliablyand accurately.

Test Cases:

Thetestcaseistheheartofatest plan. A test case is a document that describes inputs andoperatingconditionsrequired to run a test together with the expected results other run.

Thetestersshouldrunthesoftware according to the test case documentation and thencomparetheactualresults with the expected results noted in the documents Iftheobtainedresultsarein complete agreement with the expected results, thenindicatesthatnoerrorhas been identified. The result of such a test case is said to be 'pass'.

Apotentialerrorissaidtobeidentified when some or all of the results do not agree with theexpectedresults.Insuchcases, the test case will have the status 'fail'.

ObjectivesofWritingTestCases:

Verifythatthesystemmeets functional and non-functional requirements.

Detectandpreventpotential bugs, vulnerabilities, or transaction errors.

Validateintegrationwith the Google Gemini AI API and uploaded study materials.

Ensuresmoothandintuitive user experience during the CBT examination.

Testtheaccuracyofscoring, analytics, and test-history management.

Test Cases for the Identified Modules

1. Exam Configuration

Verify that users (customers, operators, and admins) can register successfully with valid credentials.

Ensure that invalid or incomplete configuration inputs are handled properly.

Check for proper error messages when invalid inputs are entered.

Validate OTP or multi-factor authentication during login.

Test password recovery and reset functionality.

2. Study Material and AI Research

Verify that PDF, DOCX, and TXT study materials can be uploaded successfully. are applied during mock-test generation.

Validate that text is extracted correctly from uploaded study material.

Ensure that AI syllabus research works correctly when requested by the aspirant.

Test appropriate error handling for invalid or unsupported study-material files.

3. AI Question Generation

Validate that the Google Gemini AI API generates questions according to the selected configuration.

Ensure that generated questions contain valid options, correct answers, explanations, and difficulty levels.

Test proper handling when AI question generation or JSON parsing fails.

4. CBT Examination and Answer Management

Verify that generated questions are displayed correctly in the CBT interface.

Ensure that aspirants can select, clear, navigate, and mark questions for review.

Validate that answers and question states are maintained correctly during the examination.

5. Timer and Submission

Verify that the countdown timer starts correctly when the examination begins.

Ensure that manual submission works correctly.

Test automatic submission when the examination timer expires.

6. Scoring and Analytics

Verify that correct, incorrect, and unanswered questions are evaluated correctly.

Ensure that negative marking is applied according to the configured penalty.

Validate that score, accuracy, section-wise analytics, and explanations are displayed correctly.

□

7. Test History Management

Verify that completed test results are saved correctly in the SQLite database.

Ensure that stored results contain exam, subject, mock type, score, accuracy, and date information

8. Performance and Load Testing

Test system responsiveness when multiple aspirants generate and attempt mock tests

Verify stable AI question generation under high load.

Ensure the system handles multiple test submissions without failure.

Test scoring and result-processing speed under high load.

9. UI/UX Testing

Verify that the CBT interface is intuitive for aspirants.

Ensure the platform is responsive and easy to use.

Check consistency in navigation, buttons, question palette, and fonts.

Validate clear display of questions, options, timer, results, and analytics.

10. Notifications and Alerts

Ensure users receive appropriate alerts for successful test submission and timer expiry.

Validate error messages for failed question generation or invalid inputs.

Test alerts for AI API failures and database errors.

□ Verify real-time timer and automatic-submission alerts.

□

□

□

Writing a Test Case for an AI Mock Test Generator

1. Aspirant is generating a mock test successfully

<i>Test Case ID</i>	<i>Test Case Name</i>	<i>Actual Output</i>	<i>Expected Output</i>
1	Valid mock test generation	Mock test generated with valid configuration	Mock test generated confirmed
2	Invalid generation request	Mock test generation failed — error message (Invalid quantity entered)	Mock test generation rejected

2. System is rejecting invalid exam configuration

<i>Test Case ID</i>	<i>Test Case Name</i>	<i>Actual Output</i>	<i>Expected Output</i>
1	Valid exam configuration within limit	Mock test generated with valid configuration	Mock test generated successfully
2	Invalid exam configuration	Test generation failed with error (Exceeds daily limit)	Generation rejected

3. AI Question Generation

<i>Test Case ID</i>	<i>Test Case Name</i>	<i>Actual Output</i>	<i>Expected Output</i>
1	Valid question generation	Questions generated successfully	Questions generated successfully
2	Invalid/Expired	Question generation with error (Expired card)	Questions generation rejected

4. CBT Examination and Scoring

<i>Test Case ID</i>	<i>Test Case Name</i>	<i>Actual Output</i>	<i>Expected Output</i>
1	Successful test submission working	Test completed, score displayed	Score and result displayed
2	Unanswered questions handled correctly	Test submitted	Unanswered questions handled correctly

□

Conclusion:

This experiment successfully demonstrated the use of test cases for different modules different testing needs. The comparison highlighted the strengths and limitations of each tool, helping in the selection of the most appropriate tool for specific testing requirements. Testing tools play a crucial role in improving software quality, reducing manual effort, and ensuring application security and reliability.

Practical No. 10

AIM:-Demonstrating the Use of Different Testing Tools with Comparison

Introduction:

Software testing is a crucial step in the Software Development Life Cycle (SDLC) that ensures an application meets quality, security, and performance standards before deployment. With the growing complexity of software systems, various testing tools are employed to automate processes, identify vulnerabilities, and enhance efficiency. This experiment explores different testing tools, comparing their features, usability, and effectiveness in achieving software quality assurance.

Objectives:

- To understand the role of different testing tools in software quality assurance.
 - To explore functional testing tools like Selenium and Test Complete.
 - To analyse performance testing tools such as JMeter and LoadRunner.
 - To examine security testing tools like OWASP ZAP and Burp Suite.
 - To compare testing tools based on ease of use, reporting capabilities, integration, and cost.
- Theory: Software testing is an essential phase in the ***Software Development Life Cycle (SDLC)*** to ensure application functionality, performance, security, and reliability.

Different types of testing tools cater to various testing needs:

- **Functional Testing Tools:** Used to verify software functionality according to requirements. Examples: Selenium, Test Complete.
- **Performance Testing Tools:** Evaluate software speed, scalability, and stability under load. Examples: Apache JMeter, LoadRunner.
- **Security Testing Tools:** Identify vulnerabilities in web applications and APIs. Examples: OWASP ZAP, Burp Suite.

Automation Testing Tools: Automate test execution to enhance efficiency. Examples: Selenium, Katalon Studio. Each testing tool provides unique features that help in improving software reliability while reducing manual efforts.

Testing Tools and Comparison

Tool Name	Type	Features	Pros	Cons
Selenium	Functional, Automation	Web Testing, UI Automation	Open-Source, supported	programming knowledge
Test Complete	Functional, Automation	UI Testing, Keyword-Driven	Easy – to- use , Script less automation	Paid tool
JMeter	Performance	Load & Stress Testing	Open-Source, Scalable	UI can be complex
LoadRunner	Performance	Load Testing, Cloud Support	Supports largescale testing	Expensive
OWASP ZAP	Security	Web Application Security	Free, Beginner friendly	Limited enterprise features
Burp Suite	Security	Penetration Testing	Advanced security testing	Paid tool for full features

Procedure

1. Select different testing tools based on categories: Functional, Performance, Security, and Automation.
2. Install and configure the tools on the system.
3. Create test cases for a sample web application.
4. Execute functional **tests** using Selenium and TestComplete.
5. Perform load testing using JMeter and LoadRunner.
6. Conduct security tests using OWASP ZAP and Burp Suite.
7. Compare test results based on execution time, accuracy, and ease of use.
8. Document findings and prepare a comparative analysis.

Conclusion:-

This experiment successfully demonstrated the use of *various software testing tools* for different testing needs. The comparison highlighted the strengths and limitations of each tool, helping in the selection of the most appropriate tool for specific testing requirements. Testing tools play a crucial role in improving software quality, reducing manual effort, and ensuring application security and reliability.

Practical No. 11

Aim: - Define security and quality aspects of the identified module.

Security and Quality Aspects of the Identified Module

Security and quality are essential components of any software module, ensuring its reliability, performance, and resistance to potential vulnerabilities. A well-structured module not only meets functional requirements but also adheres to security standards to safeguard against unauthorized access of data, security breaches, and cyber threats, protecting sensitive information and maintaining user trust. Similarly, quality aspects ensure that the module is efficient, maintainable, and scalable, contributing to a seamless user experience. Both security and quality aspects must be carefully considered during the development lifecycle to achieve a high-performing and secure module.

Security Aspects:

- 1. Authentication & Authorization:** Ensures only verified users can access the system, with role-based access control (RBAC) restricting functionalities based on roles.
- 2. Data Encryption:** Protects sensitive data by converting it into an unreadable format, ensuring confidentiality.
- 3. Input Validation:** Prevents SQL injection, XSS, and buffer overflow by verifying user inputs.
- 4. Secure Communication:** Uses HTTPS, SSL/TLS, and cryptographic protocols to prevent data interception.
- 5. Error Handling & Logging:** Prevents exposure of system details while logging errors securely.

6. Access Control Mechanisms: Implements permission-based access and security audits to minimize risks.

Error handling and logging mechanisms also play an important role in security. Proper error messages should not expose critical system details that attackers might exploit. Instead, errors should be logged securely while providing minimal information to end users. Continuous security audits and vulnerability assessments help identify potential risks and ensure that the module remains secure over time. Regular software updates and patches must be applied to address newly discovered security threats and maintain the module's integrity.

Quality Aspects:

- 1. Code Maintainability:** Ensures well-structured, readable, and documented code for easy debugging and modifications.
- 2. Performance Optimization:** Reduces response times, enhances efficiency, and ensures smooth system operation.
- 3. Reliability & Stability:** Guarantees consistent performance under different conditions through rigorous testing.
- 4. Usability & User Experience (UX):** Provides an intuitive and user-friendly interface for better adoption.
- 5. Testing & Debugging:** Includes unit testing, integration testing, and continuous monitoring for issue resolution.
- 6. Scalability:** Ensures the module can handle increasing workloads without significant architectural changes

Documentation and Reporting:

1. Document all security and quality measures implemented.
2. Create a report summarizing the findings, including identified vulnerabilities and their mitigation strategies, quality metrics (accuracy, response time), and recommendations for improvement.

Security and Quality Aspects — AI Mock Test Generator (CBT Exam Simulator)		
Vulnerability	Risk Level	Mitigation Strategy
Duplicate/race condition on test submission (aspirant submits answers twice)	High	Atomic SQLite transaction with a unique constraint on session_id; mark session as "submitted" server-side before accepting further answers.
Unauthorized access to another aspirant's test history or results	High	Session/user-based access control; verify session ownership on every history-fetch or result-view request.
Gemini API key exposure (hardcoded or bundled in client-side code)	High	Store the API key only in a server-side .env file; route all Gemini calls through a backend proxy, never call the API directly from the frontend.
Prompt injection via uploaded study material (malicious hidden text)	Medium	Sanitize and size-limit uploaded files; keep a fixed system-prompt boundary; validate file type before sending to Gemini.
SQL injection via raw queries on SQLite (exam config/history filters)	Medium	Use parameterized queries or an ORM (e.g. SQLAlchemy) for all database access; never concatenate user input into SQL strings.

Conclusion :-

Overall, security and quality aspects are integral to the development of a robust and efficient software module. Security measures such as authentication, encryption, input validation, and secure communication protect against potential threats and unauthorized access. Meanwhile, quality aspects such as maintainability, performance optimization, reliability, usability, and scalability contribute to the overall effectiveness and user experience of the module. By integrating these principles into the development process, organizations can ensure that their software modules meet high standards of security and quality, leading to long-term success and reliability in real-world applications.